LAPORAN PRAKTIKUM
EXCEPTION HANDLING

Modul Praktikum 7 - Pemrograman Berorientasi Objek

Diajukan untuk memenuhi salah satu tugas praktikum Pemrograman Berorientasi objek



Disusun Oleh:

Salman alfarisi firdaus - 251511030

2A D3

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2026

Bagian A - Exceptions Aren't Always Errors: Case 1 CountLetters

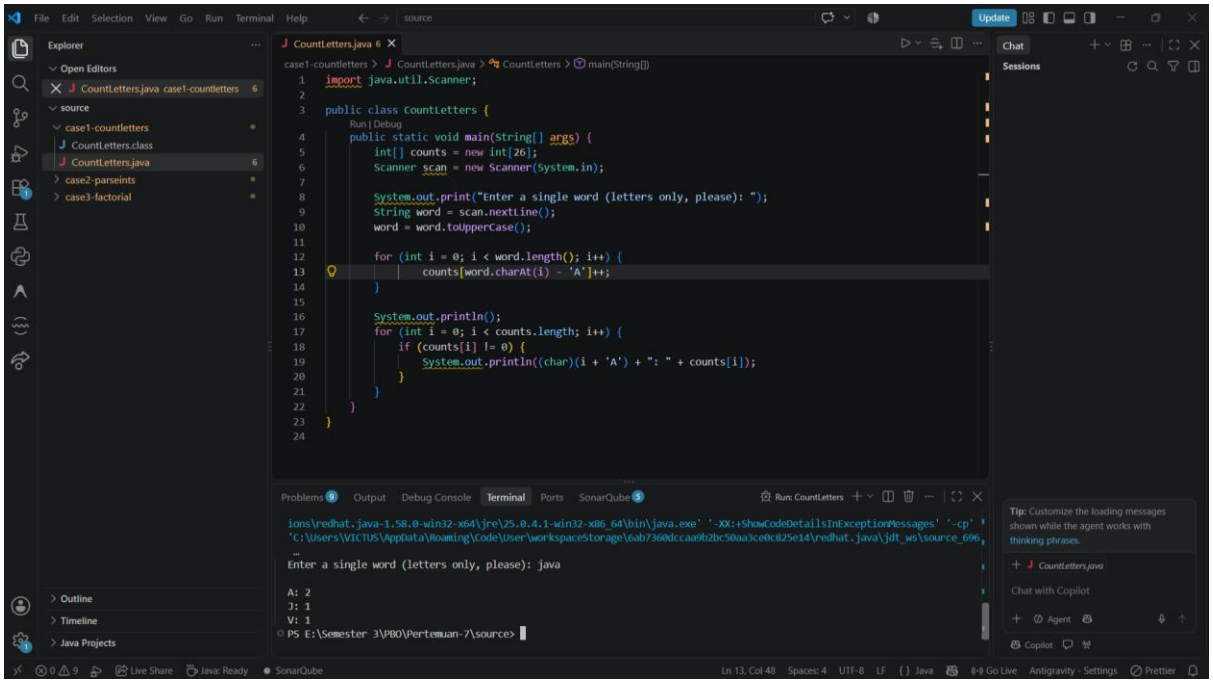
Kasus pertama membuktikan bahwa exception tidak selalu berarti aplikasi harus berhenti. Program CountLetters menghitung frekuensi huruf kapital A sampai Z dari teks masukan pengguna.

Checkpoint 1: Terjemahan Karakter dan Alokasi Array

Karakter huruf diterjemahkan ke indeks array 0 sampai 25 lewat operasi pengurangan kode ASCII:

```
counts[word.charAt(i) - 'A']++;
```

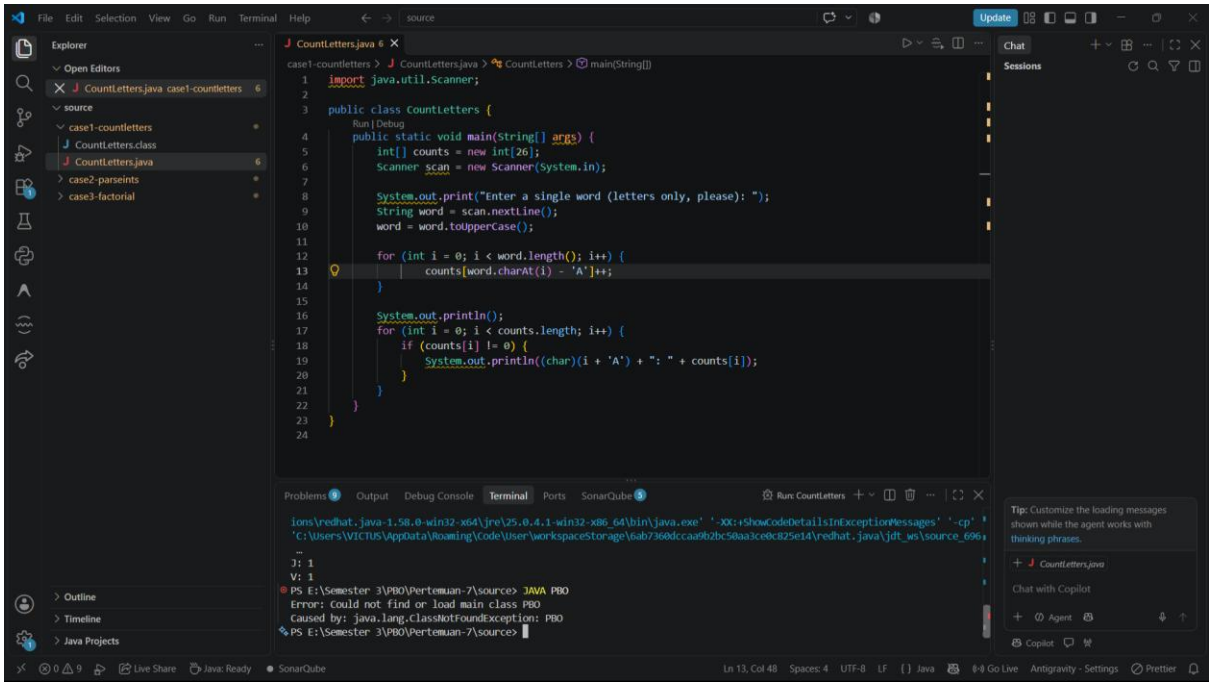
Karakter A memiliki kode ASCII 65 dan Z bernilai 90. Pengurangan dengan karakter A menghasilkan rentang 0 sampai 25. Array counts berukuran 26 karena alfabet Bahasa tepat berjumlah 26 huruf.



Step 2 dan Checkpoint 2: Uji ArrayIndexOutOfBoundsException

Saat program diberi input frasa bertanda baca seperti JAVA PBO!, program langsung berhenti akibat ArrayIndexOutOfBoundsException. Spasi bernilai ASCII 32 sehingga 32 dikurangi 65 menghasilkan indeks -33 di luar rentang array.

Exception menghentikan thread utama seketika. Akibatnya, loop pencetakan frekuensi di bawahnya tidak sempat dieksekusi.

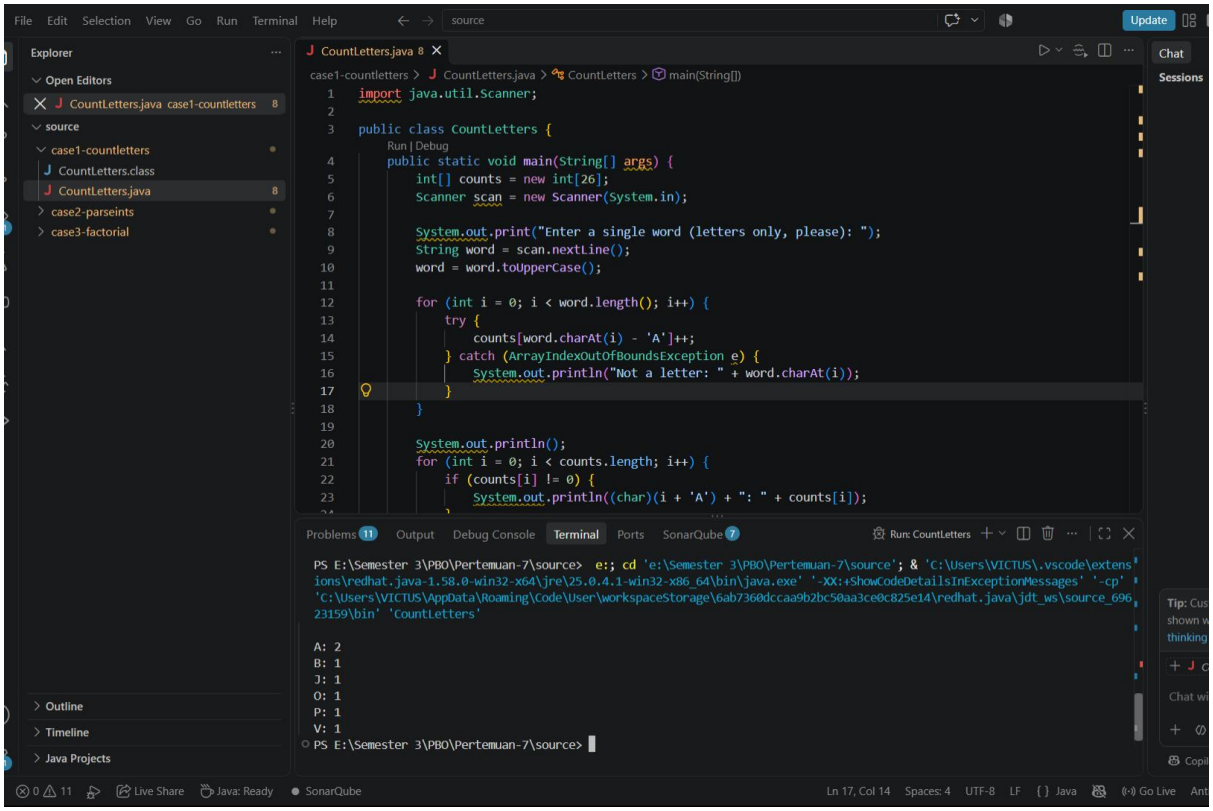


Step 3, 4 dan Checkpoint 3: Penerapan Try-Catch

Operasi penambahan frekuensi diletakkan ke dalam blok try-catch di dalam loop for:

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Not a letter: " + word.charAt(i));
    }
}
```

Sebelum ada catch, program berhenti total saat membaca spasi. Setelah diberi catch, karakter non-huruf dilewati dengan pesan peringatan, lalu loop tetap lanjut menghitung huruf berikutnya hingga selesai.





Bagian B - Placing Exception Handlers: Case 2 ParseInts

Kasus kedua menguji pengaruh letak blok try-catch terhadap alur perulangan. Program ParseInts membaca satu baris kalimat lalu menjumlahkan token angka di dalamnya.

Checkpoint 4: Penyebab NumberFormatException

Method Integer.parseInt melempar NumberFormatException saat membaca token kata seperti We karena teks tersebut bukan angka yang valid.

Step 6 dan Checkpoint 5: Try-Catch di Luar While Loop

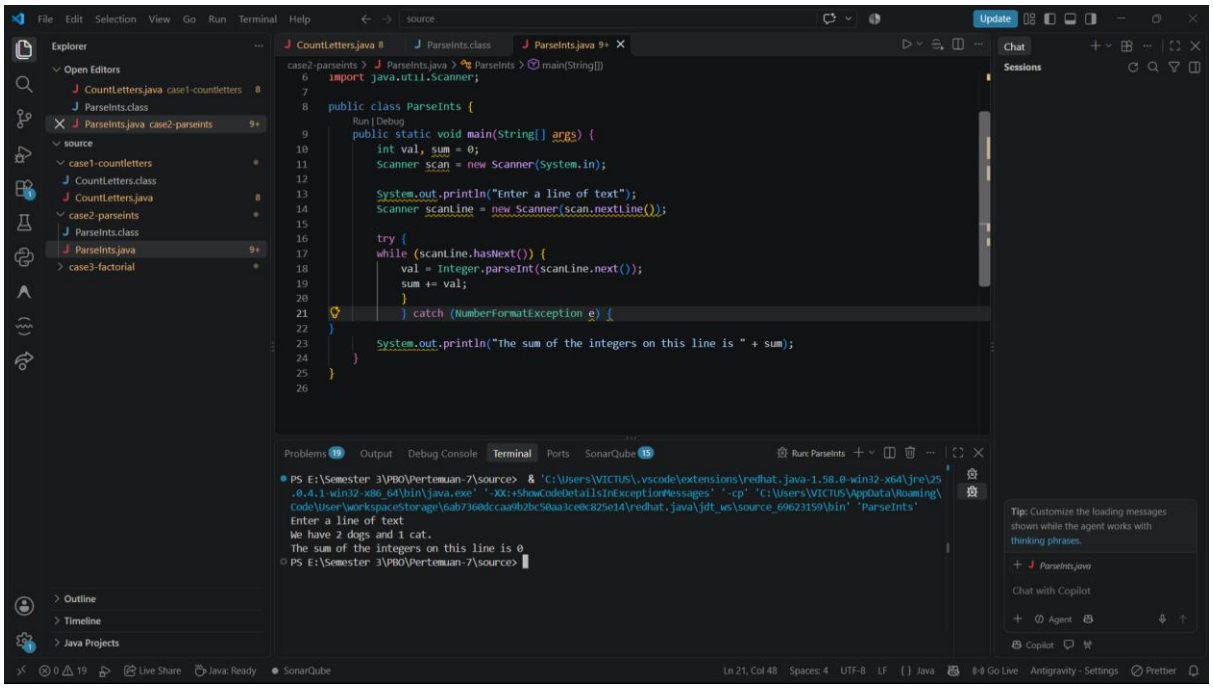
Eksperimen pertama meletakkan try-catch di luar while loop:

```
try {
    while (scanLine.hasNext()) {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    }
} catch (NumberFormatException e) {}
```

Input 'We have 2 dogs and 1 cat.' menghasilkan sum = 0, sedangkan '1 fish 2 fish' menghasilkan sum = 1.

Begitu token bukan angka ditemukan, alur program langsung terlempar keluar dari while loop menuju catch. Sisa token dalam kalimat tidak sempat diproses sama sekali.\





Step 7 dan Checkpoint 6: Try-Catch di Dalam While Loop

Pada tahap kedua, blok try-catch dipindahkan ke dalam while loop:

```
while (scanLine.hasNext()) {
    try {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    } catch (NumberFormatException e) {}
}
```

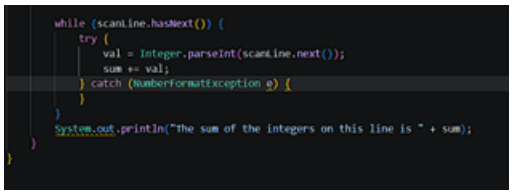
Kedua kalimat uji di atas kini menghasilkan nilai sum = 3 secara tepat.

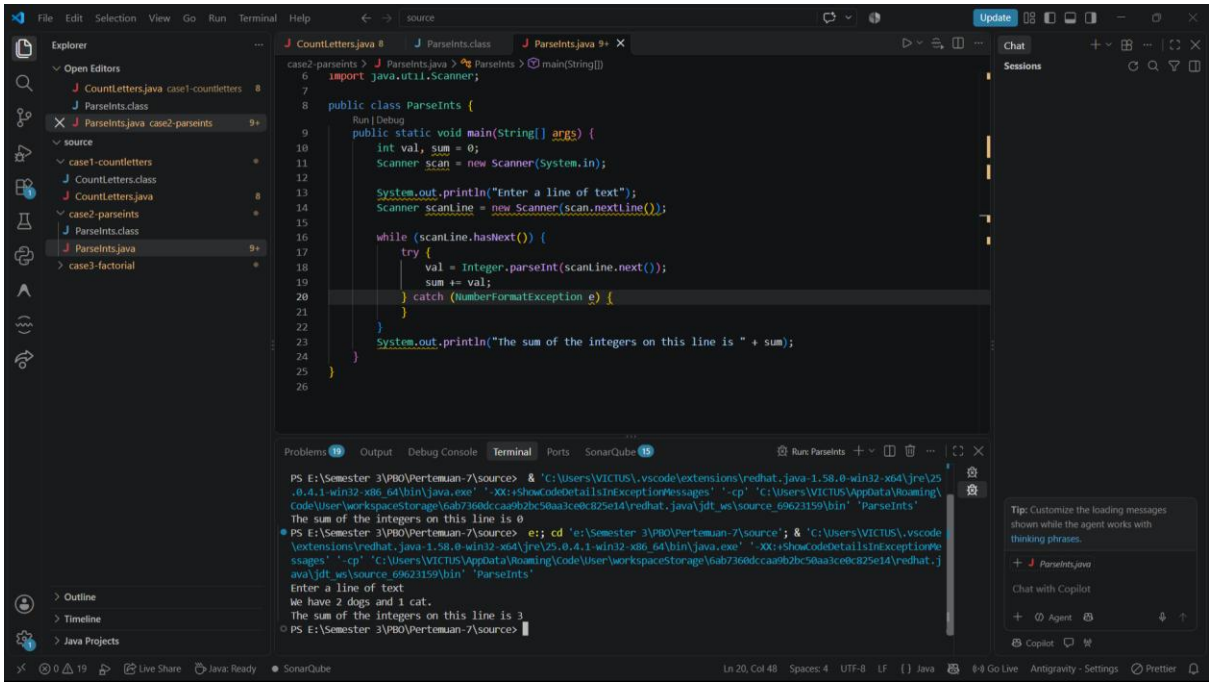
Letak handler menentukan cakupan pemulihan program. Try-catch di luar loop menghentikan perulangan saat terjadi error. Sebaliknya, try-catch di dalam loop menangani error per token sehingga iterasi berikutnya tetap berjalan.

Tabel Perbandingan Letak Handler:

Di luar while loop Alur langsung melompat keluar dari perulangan.

Di dalam while loop Alur hanya melompat ke catch pada iterasi tersebut.





Bagian C - Throwing Exceptions: Case 3 Factorials dan MathUtils

Kasus ketiga mempelajari cara melempar exception secara sengaja dari dalam method untuk menolak input data yang tidak valid.

Checkpoint 7: Bahaya Nilai Dummy Return 1

Faktorial tidak terdefinisi untuk bilangan negatif. Mengembalikan nilai 1 untuk input negatif sangat berbahaya karena nilai 1 dapat disalahartikan sebagai hasil hitung yang valid. Solusi yang tepat adalah menolak input dengan melempar exception.

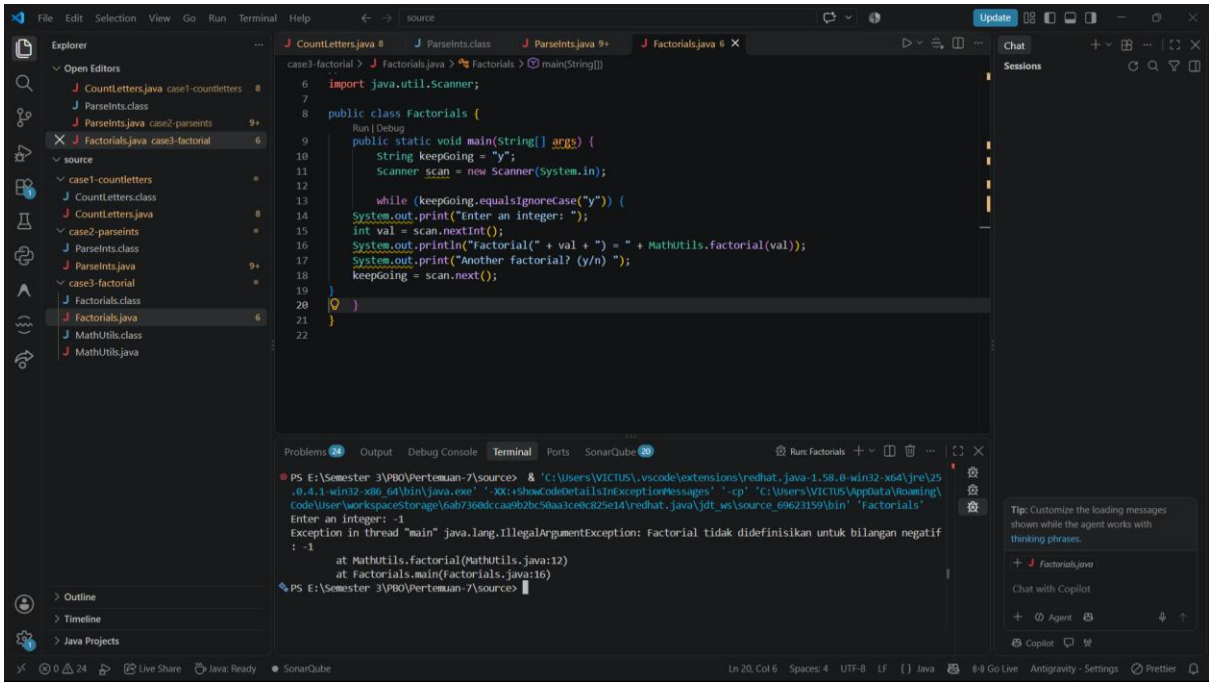
Step 10, 12 dan Checkpoint 8: Melempar IllegalArgumentException

Method factorial dimodifikasi untuk melempar exception saat menerima nilai negatif maupun nilai di atas 16 yang melampaui batas tipe data int:

```
public static int factorial(int n) throws IllegalArgumentException {
    if (n < 0) {
        throw new IllegalArgumentException("Factorial tidak terdefinisi untuk
bilangan negatif: " + n);
    }
    if (n > 16) {
        throw new IllegalArgumentException("Nilai terlalu besar untuk factorial
berbasis int: " + n);
    }
    int fac = 1;
    for (int i = n; i > 0; i--) fac *= i;
    return fac;
}
```

Method factorial bertugas mendeteksi pelanggaran batasan dan melempar exception, sedangkan method main bertugas menangkap exception dan menampilkan pesan kepada pengguna.

```java
public static int factorial(int n) throws IllegalArgumentException {
    if (n < 0) {
        throw new IllegalArgumentException("Factorial tidak didefinisikan untuk bilangan negatif: " + n);
    }
    if (n > 16) {
        throw new IllegalArgumentException("Nilai terlalu besar untuk factorial berbasis int (> 16): " +
    }
    int fac = 1;
    for (int i = n; i > 0; i--) {
        fac *= i;
    }
    return fac;
}
```



Step 11 dan Checkpoint 9: Penanganan Exception di Main

Method main menangani exception dengan try-catch di dalam loop agar program tidak mati:

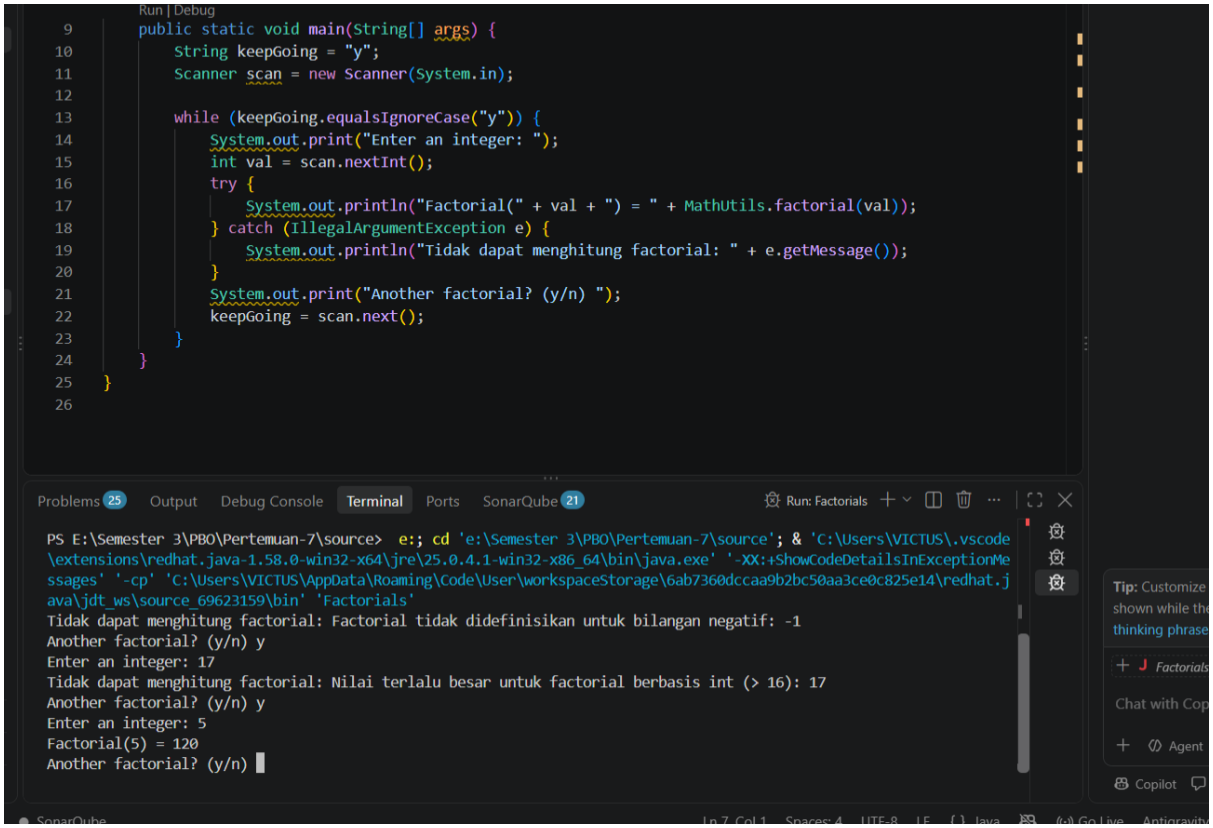
```java
while (keepGoing.equalsIgnoreCase("y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();
    try {
        System.out.println("Factorial(" + val + ") = " + MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Tidak dapat menghitung factorial: " + e.getMessage());
    }
    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```

Penggunaan e.getMessage membantu pengguna mengetahui secara spesifik apakah error disebabkan oleh nilai negatif atau karena nilai melebihi batas 16.

```java
        while (keepGoing.equalsIgnoreCase("y")) {
            System.out.print("Enter an integer: ");
            int val = scan.nextInt();
            try {
                System.out.println("Factorial(" + val + ") = " + MathUtils.factorial(val));
            } catch (IllegalArgumentException e) {
                System.out.println("Tidak dapat menghitung factorial: " + e.getMessage());
            }
            System.out.print("Another factorial? (y/n) ");
            keepGoing = scan.next();
        }
    }
}
```

```
oblems 25  Output  Debug Console  Terminal  Ports  SonarQube 21        Run: Factorials
S E:\Semester 3\PBO\Pertemuan-7\source> & 'C:\Users\VICTUS\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25
0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\VICTUS\AppData\Roaming\
ode\User\workspaceStorage\6ab7360dccaa9b2bc50aa3ce0c825e14\redhat.java\jdt_ws\source_69623159\bin' 'Factorials'
S E:\Semester 3\PBO\Pertemuan-7\source>  e:; cd 'e:\Semester 3\PBO\Pertemuan-7\source'; & 'C:\Users\VICTUS\.vscode
extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMe
sages' '-cp' 'C:\Users\VICTUS\AppData\Roaming\Code\User\workspaceStorage\6ab7360dccaa9b2bc50aa3ce0c825e14\redhat.j
va\jdt_ws\source_69623159\bin' 'Factorials'
nter an integer: -1
idak dapat menghitung factorial: Factorial tidak didefinisikan untuk bilangan negatif: -1
nother factorial? (y/n) y
nter an integer: 17
idak dapat menghitung factorial: Nilai terlalu besar untuk factorial berbasis int (> 16): 17
nother factorial? (y/n)
```


```java
while (keepGoing.equalsIgnoreCase("y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();
    try {
        System.out.println("Factorial(" + val + ") = " + MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Tidak dapat menghitung factorial: " + e.getMessage());
    }
    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```



Bagian D - Sintesis, Pengujian Skenario, dan Refleksi

1. Tabel Skenario Pengujian Modul

ID	Skenario Uji	Expected Result	Status	Keterangan
T7-01	CountLetters kata valid	Frekuensi huruf tercetak tepat	**PASS**	Input JAVA menghasilkan frekuensi benar.
T7-02	CountLetters non-huruf sebelum catch	Muncul ArrayIndexOutOfBoundsException	**PASS**	Program berhenti pada spasi dengan stack trace.
T7-03	CountLetters setelah try-catch	Karakter non-huruf dilewati dan proses lanjut	**PASS**	Pesan peringatan tampil dan huruf lain terhitung.
T7-04	ParseInts try-catch di luar while	Berhenti pada token invalid pertama	**PASS**	Kalimat campuran berhenti di kata We dengan sum 0.
T7-05	ParseInts try-catch di dalam while	Seluruh token diproses dan angka dijumlahkan	**PASS**	Kedua kalimat campuran menghasilkan sum 3.
T7-06	factorial(-1) sebelum	IllegalArgumentException	**PASS**	Program terhenti di baris throw

	catch	menghentikan program		MathUtils.
T7-07	factorial(-1) setelah catch di loop	Pesan tampil dan program tetap lanjut	**PASS**	Menampilkan pesan error dan meminta input ulang.
T7-08	factorial(17) batas overflow int	Exception nilai > 16 tertangani	**PASS**	Pesan batas kapasitas tampil dan loop berlanjut.

2. Jawaban Soal Refleksi Praktikum

1. Pada CountLetters, mengapa program awal berhenti saat menemukan karakter non-huruf? Apa yang berubah setelah ada catch?

Program awal berhenti karena spasi menghasilkan indeks negatif -33 sehingga memicu ArrayIndexOutOfBoundsException yang langsung mematikan thread utama. Setelah diberi catch, exception dinetralkan di tingkat lokal perulangan sehingga loop dapat lanjut memproses huruf berikutnya.

2. Apakah exception selalu berarti aplikasi harus berhenti? Gunakan satu bukti dari praktikum.

Tidak. Exception adalah sinyal kondisi khusus. Jika ditangani dengan try-catch, aplikasi dapat pulih dan terus berjalan. Buktinya pada ParseInts: token bukan angka dilewati tanpa membuat program crash, dan angka-angka yang ada tetap berhasil dijumlahkan.

3. Mengapa try-catch di luar while pada ParseInts menghasilkan behavior berbeda dengan try-catch di dalam while?

Try-catch di luar while memindahkan alur keluar dari loop secara permanen saat error pertama muncul. Sedangkan try-catch di dalam while hanya menangani error pada token tersebut, sehingga perulangan tetap berlanjut memeriksa token berikutnya.

4. Dalam kasus Factorials, mengapa melempar exception lebih aman daripada mengembalikan nilai yang terlihat valid tetapi sebenarnya salah?

Mengembalikan nilai palsu seperti 1 untuk input negatif sangat berbahaya karena pemanggil akan menganggap nilai tersebut sebagai hasil komputasi yang benar. Melempar exception secara tegas memberi tahu pemanggil bahwa data tidak sah dan komputasi tidak dapat dilakukan.

5. Apa perbedaan peran throw dan catch berdasarkan code yang Anda jalankan?

Keyword throw digunakan oleh method komputasi untuk mendeteksi kesalahan aturan dan melempar exception. Sedangkan catch digunakan oleh pihak pemanggil untuk menangkap exception tersebut dan memulihkan alur program.

6. Apa manfaat memberikan message berbeda pada IllegalArgumentException untuk nilai negatif dan nilai > 16?

membedakan apakah kesalahan akibat input negatif atau karena batas kapasitas tipe data int terlampaui. Hal ini mempermudah pengguna memahami error dan mempercepat debugging.

7. Ketiga exception utama pada modul ini termasuk unchecked exception. Apa yang ingin Anda tanyakan ketika nanti membandingkannya dengan checked exception?

kapan sebuah kondisi kegagalan harus dijadikan checked exception yang mewajibkan try-catch saat kompilasi, dan kapan cukup menjadi unchecked exception seperti turunan RuntimeException?

8. Lengkapi kalimat: 'Exception handling bukan sekadar mencegah program crash, tetapi ...'

Exception handling bukan sekadar mencegah program crash, tetapi cara menjaga integritas data sistem, menegakkan kontrak aturan method, dan menyediakan mekanisme pemulihan alur program saat terjadi kondisi abnormal.

Bagian E – Repository

https://github.com/salfffariss/PBO-7-ExceptionHandling